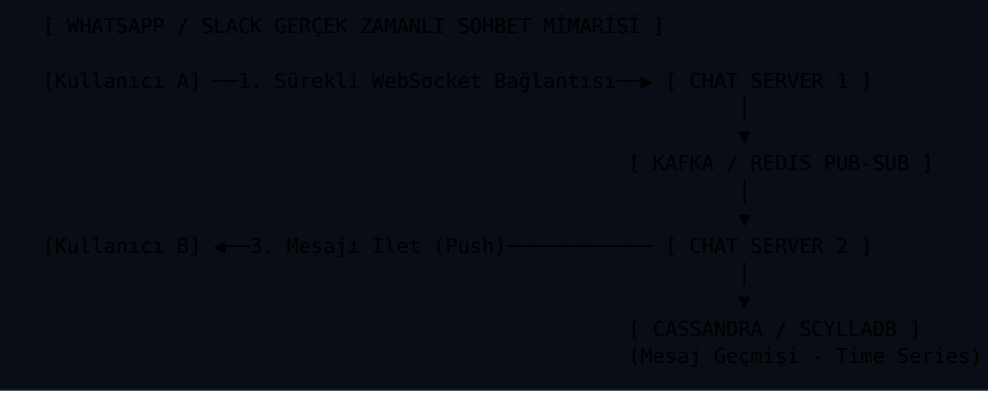
43

Gerçek Zamanlı Sohbet Sistemi (WhatsApp / Slack): WebSockets, Presence ve Cassandra

Designing a Chat System (WhatsApp / Slack): WebSockets & Presence

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski internet sitelerinde bir arkadaşınızın size mesaj yazıp yazmadığını görmek için sayfayı her 3 saniyede bir 'Yenile' (Polling) yapardınız (kapıyı 100 kez açıp kapatmak gibi). WhatsApp ve Slack ise iki ev arasına çekilmiş kesintisiz bir telefon hattı gibidir (WebSocket). Hat bir kez açılır ve 24 saat boyunca açık kalır. Arkadaşınız 'Selam' dediği anda ses kulağınızda yankılanır. 'Çevrimiçi / Online' göstergesi ise kapının önündeki lambadır: Siz odaya girdiğinizde lamba yanar (Presence Server); 30 saniye boyunca hareket etmezseniz lamba otomatik söner!

🇬🇧 ENGLISH (ELI5)

In early web chat rooms, checking for new messages meant mashing the 'Refresh' button every 3 seconds (HTTP Polling)—like constantly slamming your front door open and shut. Modern chat apps like WhatsApp and Slack deploy a persistent, open fiber-optic intercom (WebSocket connection). The duplex socket opens once and stays active. When a friend types 'Hi', the byte stream transmits instantly. The 'Online' status indicator is a motion sensor: when your phone pings the presence gateway, the light glows green; if no heartbeat arrives for 30 seconds, it shifts to offline!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Sohbet Mimarisi Bileşenleri: 1) İletişim Protokolü: İki yönlü, düşük gecikmeli, başlık yükü minimum WebSocket protokolü kullanılır. 2) Durum Takibi (Presence Service): Kullanıcının çevrimiçi/çevrimdışı durumunu yönetir. İstemciler her 10-15 saniyede bir hafif Heartbeat (Canlılık sinyali) gönderir. Heartbeat kesilirse Redis'teki oturum süresi (TTL) dolar ve kullanıcı 'Offline' işaretlenir. 3) Mesaj Depolama (Neden Cassandra?): Sohbet mesajları zaman serisi (Time-series) verisidir; yazma hacmi aşırı yüksektir, eski mesajlar nadiren güncellenir. Cassandra'nın LSM-Tree mimarisi saniyede milyonlarca mesaj yazmayı mükemmel kaldırır. Bölümleme: `Partition Key: chat_id, Clustering Key: message_id (Snowflake)` ile mesajlar kronolojik sırada diske dökülür.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Chat System Topology: 1) Transport: Full-duplex stateful WebSockets over TCP, avoiding HTTP connection handshake penalties. 2) Presence Cluster: Tracks ephemeral user heartbeats. Clients dispatch heartbeat pings every 15s. Absence of pings expires a Redis lease, broadcasting offline status to user contact channels. 3) Storage Engine (Why Cassandra / ScyllaDB?): 1-on-1 chat history exhibits append-only sequential access patterns with billions of writes per day. Relational databases choke on index rebalancing. Cassandra maps `Partition Key = conversation_id` with `Clustering Key = message_id (Snowflake time-ordered)`, storing millions of chronological messages sequentially on disk for lightning-fast pagination.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: WebSocket Oturum Eşleme ve Chat Yönlendirici Modeli
import java.util.concurrent.*;

public class ChatSessionRouter {
    // Aktif WebSocket sunucuları haritası: UserID -> Hangi Chat Server'a bağlı?
    private final ConcurrentMap<String, String> userServerRegistry = new ConcurrentHashMap<>();

    public void registerSession(String userId, String serverId) {
        userServerRegistry.put(userId, serverId);
        IO.println("Kullanıcı " + userId + " bağlandı -> Sunucu: " + serverId);
    }

    public String routeMessage(String recipientId) {
        String server = userServerRegistry.get(recipientId);
        if (server != null) {
            return "FORWARD_WEBSOCKET -> " + server;
        }
        return "DISPATCH_APNS_OR_FCM_PUSH (Kullanıcı Çevrimdışı!)";
    }
}

void main() {
    var router = new ChatSessionRouter();
    router.registerSession("user_ali", "chat-gateway-pod-1");

    IO.println("Mesaj Yönlendirme (Ali):  " + router.routeMessage("user_ali"));
    IO.println("Mesaj Yönlendirme (Ayşe): " + router.routeMessage("user_ayse"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 100.000 kişilik bir grupta biri mesaj attığında 'Mesajı gruptaki 100.000 kişinin WebSocket soketine tek tek yazarım' demeyin! Bu 'Fan-out' darbesi sunucuyu kilitler. Büyük gruplarda mesajlar merkezi bir kuyruğa yazılır ve istemciler kendi pencereleri açıldığında mesajları çeker.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Pushing group messages immediately to all 100,000 members of a mega-channel concurrently over WebSockets introduces a fatal fan-out write storm that saturates network buffers. Large channels require pull-based pagination or aggregated pub/sub partitions.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- WebSocket sürekli açık kalan iki yönlü TCP kanalıyla anlık sohbetin omurgasıdır.
- Kullanıcı durumu (Presence) periyodik Heartbeat ve Redis TTL anahtarlarıyla yönetilir.
- Sohbet geçmişi için Cassandra/ScyllaDB LSM-Tree mimarisiyle en yüksek yazma hızını sunar.

🎯 Key Takeaways for the Interview

- Persistent WebSockets eliminate handshake overhead for sub-50ms message delivery.
- Ephemeral user presence states are managed via heartbeat loops and auto-expiring Redis keys.
- Cassandra's sequential append-only LSM-Tree storage is the gold standard for chat persistence.